

Munder Difflin Multi-Agent System — Report

1. System Overview

The system uses **5 agents** built on the `smolagents ToolCallingAgent` framework to automate paper supply order processing. An LLM-based `ParsingAgent` handles natural language understanding, while business logic agents remain deterministic.

Architecture (5 Agents)

Agent	Role
Orchestrator	Coordinates workflow, routes to sub-agents, builds response
ParsingAgent	Extracts items/quantities from natural language, matches to catalog
InventoryAgent	Checks stock, evaluates reorder feasibility
QuotingAgent	Generates quotes with pricing and discounts
OrderingAgent	Commits sales and stock transactions to database

Workflow

Customer Request → Orchestrator

1. `ParsingAgent.parse_request()` → `ParsedRequest` (LLM extracts items)
 2. `InventoryAgent.assess_request()` → `can_fulfill` / `blocked`
 3. `QuotingAgent.generate_quote()` → `line items`, `discount`, `total`
 4. `OrderingAgent.finalize_order()` → `DB transactions`
- "Order confirmed" or denial with reasons

Workflow Explanation & Design Rationale

The architecture separates concerns into five single-responsibility agents, each owning one stage of the order pipeline:

1. **Orchestrator** — Acts as the central coordinator. It receives the raw customer request, delegates to each downstream agent in sequence, and assembles the final response. By keeping orchestration logic in a dedicated agent, the workflow is easy to modify (e.g., adding a new validation step) without touching individual agent implementations.
2. **ParsingAgent** — The only agent that relies on an LLM. Customer requests arrive as free-form natural language with varied phrasing (“500 sheets of glossy”, “10K A4”, “printer paper”). Rather than maintaining brittle regex patterns or synonym dictionaries, the system delegates parsing to an LLM that receives the full product catalog and returns structured item/quantity pairs. This design choice trades determinism for flexibility: the agent handles synonyms, abbreviations, and novel phrasing without code changes.

3. **InventoryAgent** — Performs deterministic stock and feasibility checks. For each parsed item, it queries the database to verify stock levels, checks whether the company’s cash balance supports a restock order, and compares supplier ETAs against the customer’s delivery deadline. Separating inventory logic lets it be tested and reasoned about independently.
4. **QuotingAgent** — Computes pricing using catalog unit prices and applies volume-based discounts. It also reviews historical quote data for consistency. Isolating quoting from ordering ensures that a quote can be generated and inspected before any money changes hands.
5. **OrderingAgent** — The only agent with database write access. It commits sale transactions and, if needed, stock replenishment orders. By restricting writes to a single agent, the system minimises the risk of inconsistent state from partial failures.

This sequential pipeline was chosen over a parallel or fully autonomous multi-agent design because each step depends on the output of the previous one (you cannot quote items that haven’t been parsed, or finalise an order that hasn’t been assessed). The strict ordering also makes the system easier to debug — failures can be traced to the exact stage that produced unexpected output.

2. Evaluation Results

Summary (20 test requests)

Metric	Value
Orders Fulfilled	10 (50%)
Partially Blocked	4 (20%)
Fully Denied	5 (25%)
Parse Failures	1 (5%)
Starting Cash	\$45,059.70
Final Cash	\$44,995.38
Starting Inventory	\$4,940.30
Final Inventory	\$4,641.25

Correctness Analysis Per Request

Req	Requested Items	LLM Parsed	Correctness
1	200 A4 glossy, 100 cardstock, 100 colored	Glossy(200), Card-stock(100), Colored(100)	Perfect — all 3 items correctly parsed

Req	Requested Items	LLM Parsed	Correct Notes
2	500 poster paper, 300 streamers, 200 balloons	Colored(500), Streamers(300)	“colorful poster paper” → Colored paper (debatable); balloons correctly skipped
3	10K A4, 5K A3, 500 printer paper	A4(15000), Standard copy(500)	A3 merged into A4 (no A3 in catalog — reasonable)
4	500 recycled cardstock, 250 A4 printer	Recycled(500), A4(250)	Correct
5	500 colored, 300 cardstock, 200 washi tape	Colored(500), Cardstock(300)	Missing 200 washi tape
6	500 construction, 300 printer, 200 cardstock	Construction(500), Card-stock(200), A4(300)	Correct
7	500 glossy, 1000 matte, 300 poster 24x36, 200 cardstock	[] (empty)	Total parse failure — LLM returned no items
8	500 glossy, 1000 matte, 2000 colored, 3000 recycled	All 4 correct	Perfect
9	200 A4, 100 glossy, 50 kraft envelopes	A4(200), Glossy(100), Kraft paper(50)	“kraft paper envelopes” → Kraft paper (should be Envelopes)
10	500 glossy, 300 cardstock	Glossy(500), Cardstock(300)	Perfect
11	500 glossy, 300 matte	Glossy(500), Matte(300)	Perfect
12	200 cardstock, 500 printer paper, 100 napkins	Cardstock(200), Std copy(500), Napkins(100)	“standard printer paper” correctly mapped to Standard copy paper
13	500 A4, 200 cardstock	A4(500), Cardstock(200)	Perfect
14	5000 A4, 2000 poster paper, 500 cardstock	A4(5000), Colored(2000), Cardstock(500)	“poster paper in various colors” → Colored paper (should be Poster paper)
15	10K A4, 5K A3 colored, 500 cardboard	A4(10000), Colored(5000)	“cardboard for signage” correctly skipped

Req	Requested Items	LLM Parsed	Correct Notes
16	500 A4, 200 construction, 100 poster board	A4(500), Colored(200), Poster(100)	“colorful construction paper” → Colored paper (should be Construction paper)
17	1000 A4, 500 colored, 2000 napkins, 1000 cups, 500 plates	All 5 correct	Perfect — complex multi-item request handled well
18	500 cardstock, 1000 printing paper, 200 colored	Cardstock(500), A4(1000), Colored(200)	Correct
19	2000 glossy, 1500 matte, 1000 cardstock	All 3 correct	Perfect
20	5000 flyers, 2000 posters, 10000 tickets	Flyers(5000), Poster(2000)	Missing 10000 tickets (should be Invitation cards)

Accuracy Summary

Category	Count	Rate
Fully correct parsing	12	60%
Minor mapping issue	7	35%
Total parse failure	1	5%

3. Strengths

The following strengths are supported by specific evidence from test_results.csv:

1. **No duplicate items** — Across all 20 requests in test_results.csv, every fulfilled order contains unique line items only. For example, Request 8 correctly produces exactly four distinct items (Glossy, Matte, Colored, Recycled) with no overlap.
2. **Flexible synonym handling** — The system correctly resolves informal product names without manual rules. In test_results.csv, Request 12 maps “printer paper” to Standard copy paper, Request 4 maps “recycled cardstock” to Recycled paper, and Request 6 maps “printer” to A4 paper — all without hard-coded synonyms.
3. **Accurate bulk discount application** — test_results.csv shows discounts are consistently applied to qualifying orders: Request 3 receives a \$92.40 discount on a \$770 subtotal (12%), Request 8 gets \$86.40 off \$720

(12%), and Request 20 receives \$150 off \$1,250 (12%). Smaller orders like Request 1 (\$65 subtotal) correctly receive no discount.

4. **Correct denial handling** — The system properly denies orders when constraints are violated. Requests 13, 15, and 18 in `test_results.csv` are fully denied with specific supplier ETA vs. delivery date explanations. Requests 9, 14, 16, 17, and 19 produce partial denials with item-level reasons — no silent failures.
5. **Financial integrity** — Cash and inventory values in `test_results.csv` track correctly across all 20 rows. Cash decreases only for stock replenishment (e.g., dropping from \$45,162.93 to \$44,995.38 between Requests 19 and 20), while fulfilled orders increase cash via sales revenue. The final state (\$44,995.38 cash, \$4,641.25 inventory) is consistent with the transaction history.

4. Weaknesses

1. **LLM non-determinism** — Request 7 returned empty despite having valid items. Re-running may produce different results.
2. **Missed items** — Requests 5 (washi tape) and 20 (tickets) had items dropped by the LLM.
3. **Ambiguous mapping** — “colorful poster paper” and “poster paper in various colors” sometimes map to Colored paper instead of Poster paper.
4. **Added latency** — Each request now takes 2-4s for the LLM parsing call.
5. **No partial fulfillment** — Still all-or-nothing when items are blocked.

5. Suggested Improvements

1. **Partial fulfillment with customer confirmation** — Currently, if any item in a multi-item order is blocked (e.g., supplier ETA misses the delivery date), the entire order is denied. As seen in `test_results.csv` Requests 14, 16, 17, and 19, the system could instead fulfill the available items immediately and offer the customer the option to back-order blocked items at a later delivery date. This would increase fulfillment rate and customer satisfaction without compromising inventory accuracy.
2. **Retry mechanism for LLM parse failures** — Request 7 resulted in a total parse failure (empty item list) despite containing four valid catalog items. Adding a single automatic retry with a rephrased prompt — or falling back to a simpler keyword-matching heuristic — would eliminate this failure mode. The retry could also include the previous empty response as negative feedback to the LLM, improving the chance of a correct second attempt.
3. **Confidence scoring and human escalation** — For ambiguous mappings (e.g., “colorful poster paper” → Colored paper vs. Poster paper, seen in Requests 2, 14, and 16), the `ParsingAgent` could attach a confidence score to each item mapping. Items below a threshold (e.g., 80%) would

be flagged for human review before the order proceeds, reducing incorrect substitutions while keeping the automated path fast for high-confidence requests.